

Experiment No: 01

Experiment name: Write a Prolog program to print a list of items.

Objectives

To learn how lists work in Prolog and understand the head and tail concept. Also, to write a simple recursive program that can print all the items in a list one by one.

Theory

In normal programming languages like C or Python, we use **for** or **while** loops to print a list. But Prolog is a logic programming language, so it doesn't have traditional loops. Instead, it relies on recursion and pattern matching.

In Prolog, a list is written inside square brackets. Every list (except an empty one) has two parts: a Head and a Tail. The Head is just the first item, and the Tail is the rest of the list. We use the $[H|T]$ operator to separate them.

To print all the items, our program does two things:

- **Recursive Step:** It takes the list and splits it. It prints the Head, prints a space, and then passes the Tail back into the same function to repeat the process.
- **Base Case:** The program needs to know when to stop. When the list is finally empty ($[]$), the program hits the base case and stops running.

Code

```
print_list([]).  
print_list([H|T]) :- write(H), write(' '), print_list(T).
```

Query and Output

```
?- print_list([apple, banana, cherry, date]).  
apple banana cherry date  
true.
```

Conclusion

In this experiment, I successfully wrote a Prolog program to print the elements of a list. I learned how recursion is used instead of traditional loops in Prolog. Splitting the list into head and tail made it very easy to go through the items step by step until the list was completely empty.

Experiment No: 02

Experiment name: Write a Prolog program to find maximum between two numbers.

Objectives

To learn how to compare numbers in Prolog using mathematical operators. Also, to understand how to write multiple rules to act like an if-else statement to find the larger of two given numbers.

Theory

In normal programming languages, finding the maximum number is usually done with an **if-else** statement. But Prolog is rule-based, so we don't use standard conditional blocks. Instead, we write multiple rules with the same name and let Prolog test them to see which one is true.

To find the maximum between two numbers (let's call them X and Y), we need to write two separate rules:

- **First Rule:** This rule checks if X is the maximum. It tests if X is greater than or equal to Y. If this is true, Prolog sets X as the final answer.
- **Second Rule:** This rule checks if Y is the maximum. It tests if X is less than Y. If the first rule failed, Prolog checks this one. If true, it sets Y as the final answer.

Prolog uses standard math operators for this, like `>=` (greater than or equal to) and `<` (less than). When we run a query, the Prolog engine goes through these conditions top-to-bottom until it finds the match.

Code

```
find_max(X, Y, X) :- X >= Y.  
find_max(X, Y, Y) :- X < Y.
```

Query and Output

```
?- find_max(15, 25, Max).  
Max = 25.
```

```
?- find_max(40, 10, Max).  
Max = 40.
```

Conclusion

In this experiment, I successfully wrote a Prolog program to find the maximum of two numbers. I learned how to use relational operators inside Prolog rules. Writing two separate rules for the same function showed me how Prolog evaluates conditions logically step-by-step, making it very easy to compare values without needing traditional loops or conditional keywords.

Experiment No: 03

Experiment name: Write a Prolog program to find the GPA of a student.

Objectives

To learn how to use multiple conditions to categorize numbers in Prolog. Also, to understand how to use the logical AND operator to define specific number ranges for grading a student's marks.

Theory

In normal programming languages, calculating a grade is usually done using long **if-else** if chains to see where a student's score lands. In Prolog, we don't use if-statements. Instead, we define several rules with the same name, and Prolog checks them one by one from top to bottom until it finds a match.

To figure out a GPA, we need to check if a mark falls within a specific range (for example, between 70 and 74). In Prolog, we can combine two conditions using a comma, which acts as a logical AND. This means both conditions must be true for the rule to succeed. By writing a list of rules that cover all possible mark ranges—from a perfect 4.00 down to a failing 0.00—we can give Prolog a mark, and it will figure out the matching GPA by testing the rules until one fits.

Code

```
gpa(Mark, 4.00) :- Mark >= 80.  
gpa(Mark, 3.75) :- Mark >= 75, Mark < 80.  
gpa(Mark, 3.50) :- Mark >= 70, Mark < 75.  
gpa(Mark, 3.25) :- Mark >= 65, Mark < 70.  
gpa(Mark, 3.00) :- Mark >= 60, Mark < 65.  
gpa(Mark, 2.75) :- Mark >= 55, Mark < 60.  
gpa(Mark, 2.50) :- Mark >= 50, Mark < 55.  
gpa(Mark, 2.25) :- Mark >= 45, Mark < 50.  
gpa(Mark, 2.00) :- Mark >= 40, Mark < 45.  
gpa(Mark, 0.00) :- Mark < 40.
```

Query and Output

```
?- gpa(82, Gpa).  
Gpa = 4.0.
```

```
?- gpa(52, Gpa).  
Gpa = 2.5.
```

```
?- gpa(42, Gpa).  
Gpa = 2.0.
```

```
?- gpa(35, Gpa).
```

Gpa = 0.0.

Conclusion

In this experiment, I successfully wrote a Prolog program to calculate a student's GPA based on their marks. I learned how to use the comma operator to combine multiple conditions, which allowed me to define clear boundaries for each grade tier. It showed me how easily Prolog handles grading systems simply by matching inputs against a structured list of logical rules.

Experiment No: 04

Experiment name: Write a Prolog program that describes the student-professor relationships.

Objectives

To learn how to write basic facts in Prolog and use them to build logical rules. Also, to model a real-world scenario by connecting students, professors, and the academic courses they share.

Theory

In Prolog, a database is just a collection of known facts. For example, we can state that a certain professor teaches a specific course, and a certain student studies that same course. These are written as simple relations.

But the real power of Prolog comes from rules. Instead of manually listing out every single student a professor teaches, we can write a dynamic rule. The rule will logically state: A professor teaches a student if the professor teaches a course, and the student studies that exact same course. By using a shared uppercase variable for the course, Prolog will automatically search through all the given facts and link the correct professor to the correct student.

Code

```
teaches(DR_MAHBUBUN_NAHAR, ai).
teaches(DR_MAHBUBUN_NAHAR, algorithms).
teaches(dr_smith, database).
studies(nobin, ai).
studies(babu, ai).
studies(shafin, algorithms).
studies(dipu, database).
teaches_student(Prof, Student) :- teaches(Prof, Course), studies(Student,
    Course).
```

Query and Output

```
?- teaches_student(DR_MAHBUBUN_NAHAR, nobin).
true.
```

```
?- teaches_student(DR_MAHBUBUN_NAHAR, Student).
Student = nobin ;
Student = babu ;
Student = shafin.
```

```
?- teaches_student(Prof, dipu).
Prof = dr_smith.
```

Conclusion

In this experiment, I successfully created a Prolog database using simple facts and relationships. I learned how to connect two different pieces of information by using a common variable inside a rule. This showed me how Prolog can automatically figure out complex relationships, like exactly who teaches whom, just from a few basic data points without needing any complex programming logic.

Experiment No: 05

Experiment name: Write a Prolog program to find nth number of Fibonacci series.

Objectives

To learn how to generate a mathematical sequence using recursion in Prolog. Also, to understand how to calculate and add previous results together to find the next number in the Fibonacci series.

Theory

In normal programming languages, we usually use a ‘for’ loop to keep adding the last two numbers together to get the next Fibonacci number. In Prolog, we don’t have loops, so we rely on recursive rules instead.

The Fibonacci sequence starts with 0 and 1. These are our base cases. We write two simple facts stating that the 0th Fibonacci number is 0, and the 1st Fibonacci number is 1.

To find any other N th number, the math rule says we must add the $(N - 1)$ th number and the $(N - 2)$ th number. In Prolog, we write a rule that first calculates $N - 1$ and $N - 2$. Then, it recursively calls the Fibonacci function on both of those new numbers to get their results. Finally, it uses the ‘is’ operator to add those two results together to give the final answer. Prolog remembers all the steps in its stack and adds them up as it finishes the recursive calls.

Code

```
fibonacci(0, 0).  
fibonacci(1, 1).  
fibonacci(N, R) :- N1 is N - 1, N2 is N - 2, fibonacci(N1, R1), fibonacci(N2, R2), R is R1 +  
    R2.
```

Query and Output

```
?- fibonacci(6, Result).  
Result = 8.
```

```
?- fibonacci(10, Result).  
Result = 55.
```

Conclusion

In this experiment, I successfully wrote a Prolog program to find the N th number in the Fibonacci sequence. I learned how to use multiple recursive calls inside a single rule to calculate previous values. Using the ‘is’ operator allowed me to perform standard arithmetic addition on the results, showing how Prolog can easily handle mathematical series through logical recursion.

Experiment No: 06

Experiment name: Write a Prolog program to calculate factorial of a number.

Objectives

To learn how to use recursion to solve mathematical problems. Also, to understand how to multiply a number by the result of a previous recursive call to calculate a final product.

Theory

In normal programming languages, calculating a factorial is usually done with a loop that counts down and multiplies numbers together. Since Prolog does not have loops, we handle this using recursive rules.

The mathematical rule for a factorial says that the factorial of a number N is N multiplied by the factorial of $N - 1$. To make sure the program doesn't run forever, we need a base case. For factorials, the base case is that the factorial of 0 is exactly 1.

In Prolog, we write a rule that first checks if N is greater than 0. Then, it subtracts 1 from N and recursively calls the factorial function on that new number. Once it hits the base case of 0, it starts returning the answers back up the chain. We use the `is` operator to multiply the current number by the result of the recursive call, saving it all in a final variable.

Code

```
factorial(0, 1).  
factorial(N, Result) :- N > 0, N1 is N - 1, factorial(N1, R1), Result is N *  
    R1.
```

Query and Output

```
?- factorial(5, Result).  
Result = 120.
```

```
?- factorial(7, Result).  
Result = 5040.
```

Conclusion

In this experiment, I successfully wrote a Prolog program to calculate the factorial of a number. I learned how to use a base case to properly stop a recursive mathematical function. By using the `is` operator, I was able to multiply the current number with the result of the recursive call, proving that Prolog can easily handle cumulative math calculations without needing a traditional loop.

Experiment No: 07

Experiment name: Write a Prolog program to demonstrate the cube of a number.

Objectives

To learn how to perform basic arithmetic operations in Prolog. Also, to understand how to calculate the cube of a given number by multiplying it by itself three times and storing the answer.

Theory

In regular programming languages, we can just assign a math operation to a variable using a standard equal sign. In Prolog, math works a little differently. We have to use the **is** keyword, which tells the Prolog engine to evaluate the mathematical expression on the right side and bind the final result to the variable on the left side.

To find the cube of any number (let's call it N), the mathematical formula is simply $N \times N \times N$. In our Prolog program, we write a very simple rule that takes an input number N and an output variable for the result. Using the **is** operator, the rule multiplies N by itself three times and gives back the final cubed value.

Code

```
cube(N, Result) :- Result is N * N * N.
```

Query and Output

```
?- cube(3, Result).  
Result = 27.
```

```
?- cube(5, Result).  
Result = 125.
```

Conclusion

In this experiment, I successfully wrote a Prolog program to calculate the cube of a number. I learned how to use the **is** operator to evaluate basic mathematical expressions and bind the answer to a variable. This showed me how straightforward basic arithmetic can be inside Prolog rules without needing any extra functions or libraries.

Experiment No: 08

Experiment name: Write a Prolog program to satisfy a list of goals.

Objectives

To learn how Prolog can execute a sequence of commands stored in a list. Also, to understand the meta-calling concept where data inside a list is treated as a runnable program goal.

Theory

In normal programming, we write commands one line after another to execute them in order. In Prolog, we can actually pass a list of actions (or goals) into a rule and ask Prolog to run them one by one dynamically.

To do this, we use a special built-in command named `call`. Our program will take a list and use the standard $[H|T]$ pattern to split it into a Head and a Tail. The `call` function will execute whatever is in the Head. Then, the program will recursively send the Tail back into itself to run the next items. When the list is finally empty, the program reaches the base case and stops running.

Code

```
satisfy_goals([]).  
satisfy_goals([Goal|Rest]) :- call(Goal), satisfy_goals(Rest).
```

Query and Output

```
?- satisfy_goals([write('First '), write('Second '), write('Third')]).  
First Second Third  
true.
```

Conclusion

In this experiment, I successfully wrote a Prolog program that executes a list of goals. I learned how to use the built-in `call` function to treat simple list items as actual executable commands. This showed me how Prolog can dynamically read and run instructions step-by-step using standard list recursion.

Experiment No: 09

Experiment name: Write a Prolog program to find GCD and LCM of two numbers.

Objectives

To learn how to use mathematical formulas and recursion together in Prolog. Also, to figure out how to calculate the Greatest Common Divisor (GCD) using the Euclidean algorithm and then use that result to easily find the Least Common Multiple (LCM).

Theory

In normal math, finding the GCD means finding the biggest number that divides two numbers without leaving a remainder. A famous way to do this is the Euclidean algorithm, which keeps finding the remainder of division until the remainder is zero. In languages like C++, we would use a ‘while’ loop for this. In Prolog, we use recursion.

We write a base case that says if the second number becomes 0, the first number is our GCD. If it’s not 0, we use the ‘mod’ operator to find the remainder of the first number divided by the second. Then, we recursively pass the second number and this new remainder back into the function.

Once we know the GCD, finding the LCM is very simple. There is a standard math rule: if you multiply two numbers together and then divide by their GCD, you get the LCM. So, we can write a second rule for LCM that first calls our GCD rule to get the divisor, and then uses the ‘is’ operator to do the final multiplication and division.

Code

```
gcd(X, 0, X).  
gcd(X, Y, Result) :- Y > 0, Rem is X mod Y, gcd(Y, Rem, Result).  
lcm(X, Y, Result) :- gcd(X, Y, G), Result is (X * Y) // G.
```

Query and Output

```
?- gcd(12, 18, Gcd).  
Gcd = 6.
```

```
?- lcm(12, 18, Lcm).  
Lcm = 36.
```

```
?- lcm(15, 20, Lcm).  
Lcm = 60.
```

Conclusion

In this experiment, I successfully wrote a Prolog program to calculate the GCD and LCM of two numbers. I learned how to turn the Euclidean algorithm into a recursive Prolog rule using the modulo operator. I also saw how one rule can call another rule to get a

piece of data before finishing its own math calculation, which makes it very easy to link different formulas together.

Experiment No: 10

Experiment name: Write a Prolog program for a cyclic graph.

Objectives

To learn how to represent a graph using simple facts in Prolog. Also, to write a recursive rule that can travel through the graph's connections to detect if a cycle exists (when a path loops back to its starting point).

Theory

A graph is a structure made up of nodes connected by lines called edges. In Prolog, we can build a graph by just writing facts that say which node connects to which, like a map.

To explore the graph, we need to teach Prolog how to find a path. A path exists from point A to point B if they are directly connected by an edge. If they aren't directly connected, a path still exists if we can step to a middle node and then find a path from that middle node to the final destination. This requires recursion.

A graph is considered cyclic if it contains a loop. In logic terms, this means we can start at a specific node, follow the edges, and eventually find a path that leads right back to that exact same starting node. By writing a rule that checks if a path exists from a variable back to itself, we can ask Prolog to trace the edges and figure out if a loop is hiding in our graph.

Code

```
edge(a, b).
edge(b, c).
edge(c, a).
edge(c, d).
path(X, Y) :- edge(X, Y).
path(X, Y) :- edge(X, Z), path(Z, Y).
cycle(Node) :- path(Node, Node).
```

Query and Output

```
?- cycle(a).
true.
```

```
?- cycle(d).
false.
```

Conclusion

In this experiment, I successfully represented a directed graph in Prolog using edge facts. I learned how to write a recursive path-finding rule that navigates through these edges step-by-step. By checking if a node has a valid path back to itself, I was able to easily detect

loops, showing how Prolog's automatic backtracking is perfect for exploring connected data structures like graphs.

Experiment No: 10

Experiment name: Write a Prolog program for a cyclic graph.

Objectives

To learn how to represent a graph using simple facts in Prolog. Also, to write a recursive rule that can travel through the graph's connections to detect if a cycle exists (when a path loops back to its starting point).

Theory

A graph is a structure made up of nodes connected by lines called edges. In Prolog, we can build a graph by just writing facts that say which node connects to which, like a map.

To explore the graph, we need to teach Prolog how to find a path. A path exists from point A to point B if they are directly connected by an edge. If they aren't directly connected, a path still exists if we can step to a middle node and then find a path from that middle node to the final destination. This requires recursion.

A graph is considered cyclic if it contains a loop. In logic terms, this means we can start at a specific node, follow the edges, and eventually find a path that leads right back to that exact same starting node. By writing a rule that checks if a path exists from a variable back to itself, we can ask Prolog to trace the edges and figure out if a loop is hiding in our graph.

Code

```
edge(a, b).  
edge(b, c).  
edge(c, a).  
edge(c, d).  
path(X, Y) :- edge(X, Y).  
path(X, Y) :- edge(X, Z), path(Z, Y).  
cycle(Node) :- path(Node, Node).
```

Query and Output

```
?- cycle(a).  
true.
```

```
?- cycle(d).  
false.
```

Conclusion

In this experiment, I successfully represented a directed graph in Prolog using edge facts. I learned how to write a recursive path-finding rule that navigates through these edges step-by-step. By checking if a node has a valid path back to itself, I was able to easily detect

loops, showing how Prolog's automatic backtracking is perfect for exploring connected data structures like graphs.

Experiment No: 11

Experiment name: Write a Prolog program for file handling.

Objectives

To learn how to create, write to, and read from external text files using Prolog. Also, to understand how file streams work to save data permanently on the computer so it doesn't get lost when the program closes.

Theory

In any programming language, data is usually lost when the program stops running. To keep it, we have to save it into a file. In Prolog, we handle files by creating something called a "Stream," which is basically a pipeline connecting the Prolog engine to a text file on the hard drive.

To write to a file, we use the **open** command and tell Prolog we want to use **write** mode. This gives us a stream variable. We then use the standard **write** command, but we pass the stream variable to it so Prolog knows to send the text to the file instead of the screen. Finally, we must use **close** to shut the pipeline and safely save the file.

To read the data back, we do the exact same thing but open the file in **read** mode. We use the **read** command to pull the data from the stream into a variable. Because we are using Prolog's built-in term reader, the text we save needs to have a period (.) at the end so Prolog knows exactly where the data stops.

Code

```
write_to_file :- open('my_data.txt', write, Stream), write(Stream,
    'hello_prolog.'), close(Stream).
read_from_file :- open('my_data.txt', read, Stream), read(Stream, Data),
    write(Data), close(Stream).
```

Query and Output

```
?- write_to_file.
true.
```

```
?- read_from_file.
hello_prolog
true.
```

Conclusion

In this experiment, I successfully wrote a Prolog program to handle basic file operations. I learned how to open a file stream, write a simple term into it, and close it properly to save the changes. Reading the file back taught me how Prolog accesses external data, proving that it is very easy to store and retrieve information permanently using standard built-in stream commands.

Experiment No: 12

Experiment name: Write a Prolog program to implement bubble sort.

Objectives

To learn how to sort a list of numbers in Prolog. Also, to understand how to compare and swap adjacent items, and use recursion to repeat the process until the entire list is in the correct order.

Theory

In normal programming languages, bubble sort uses two nested ‘for’ loops to constantly check and swap items that are next to each other. Because Prolog does not have loops, we handle this by writing a recursive helper rule that acts like a single pass through the list.

The helper rule looks at the first two items of a list. If the first number is bigger than the second, it swaps their places. If it is not bigger, it leaves the first number alone and recursively checks the rest of the list.

However, one pass usually doesn’t sort the whole list. So, our main rule calls this swap function, takes the newly swapped list, and feeds it back into itself again. It keeps doing this over and over. When the list is perfectly sorted, the swap function will fail to find any numbers to swap. At that point, Prolog falls back to the base case, which simply says the list is fully sorted and stops running.

Code

```
bubble_sort(List, Sorted) :- swap(List, NewList), !, bubble_sort(NewList,
    Sorted).
bubble_sort(Sorted, Sorted).
swap([X, Y | Rest], [Y, X | Rest]) :- X > Y.
swap([Z | Rest], [Z | NewRest]) :- swap(Rest, NewRest).
```

Query and Output

```
?- bubble_sort([5, 2, 9, 1, 4], Sorted).
Sorted = [1, 2, 4, 5, 9].
```

```
?- bubble_sort([10, 3, 7, 8], Sorted).
Sorted = [3, 7, 8, 10].
```

Conclusion

In this experiment, I successfully wrote a Prolog program to sort a list using the bubble sort algorithm. I learned how to use a helper rule to compare and swap elements using pattern matching. I also saw how the cut operator (!) is used to stop Prolog from doing unnecessary backtracking once a valid swap is made, proving that recursion can easily replace complex nested loops for sorting data.

Experiment No: 13

Experiment name: Write a Prolog program to concatenate two lists.

Objectives

To learn how to combine two separate lists into a single list in Prolog. Also, to understand how to use structural recursion to break down the first list and attach its elements to the front of the second list.

Theory

In regular programming languages, combining arrays or lists is usually done by looping through the first list and pushing each item into a new array, followed by the second list. In Prolog, we do not have loops or push commands, so we use recursion and the head-tail splitting operator ($[H|T]$).

To concatenate two lists, our program needs three variables: the first list, the second list, and a third variable to hold the final combined answer.

The base case is simple: if the first list is empty ($[]$), then combining it with the second list just gives us the exact same second list.

If the first list is not empty, we split it into a Head and a Tail. We leave the Head aside for a moment and recursively pass the Tail and the second list back into the function. When the recursion finishes building the rest of the list, Prolog automatically attaches the Head back onto the front of the final result. It repeats this until the entire first list has been attached to the second one.

Code

```
concatenate([], List, List).  
concatenate([H|T], List, [H|Result]) :- concatenate(T, List, Result).
```

Query and Output

```
?- concatenate([1, 2, 3], [4, 5], Result).  
Result = [1, 2, 3, 4, 5].
```

```
?- concatenate([a, b], [c, d, e], Result).  
Result = [a, b, c, d, e].
```

Conclusion

In this experiment, I successfully wrote a Prolog program to concatenate two lists. I learned how to use structural recursion to travel through the first list item by item while keeping the second list unchanged. By building the final result list piece by piece as the recursive calls returned, I saw how Prolog dynamically constructs new data structures without needing standard loops or temporary variables.

Experiment No: 14

Experiment name: Write a Prolog program to delete an element from a list.

Objectives

To learn how to search for and remove a specific item from a list in Prolog. Also, to understand how to build a brand new list that contains everything except the deleted item using recursion.

Theory

In normal programming languages, we usually use built-in functions to easily remove an item from an array. In Prolog, lists are immutable, meaning we can't actually change or shrink an existing list. Instead, we have to build a new one piece by piece.

To delete an element, we use the head-tail splitting operator to look at the first item of the list. If the Head is the exact item we want to delete, we just ignore it and recursively call the function on the Tail. If the Head is not the item we are looking for, we keep it, attach it to our final answer, and then recursively check the Tail.

We use the `\=` (not equal) operator to make sure we only keep the items that don't match our target. We repeat this process until the list is completely empty. The base case simply states that trying to delete any item from an empty list just results in an empty list.

Code

```
delete_element(_, [], []).
delete_element(X, [X|T], Result) :- delete_element(X, T, Result).
delete_element(X, [H|T], [H|Result]) :- X \= H, delete_element(X, T, Result).
```

Query and Output

```
?- delete_element(3, [1, 2, 3, 4, 3, 5], Result).
Result = [1, 2, 4, 5].
```

```
?- delete_element(apple, [banana, apple, orange], Result).
Result = [banana, orange].
```

Conclusion

In this experiment, I successfully wrote a Prolog program to delete an element from a list. I learned how to use pattern matching to skip over a specific value while keeping the rest of the items. It showed me how Prolog creates modified lists by recursively breaking them down and building them back up without the unwanted elements, proving that logic rules can easily replace standard array manipulation methods.

Experiment No: 15

Experiment name: Write a Prolog program to insert an element into a list.

Objectives

To learn how to add a new item into an existing list in Prolog. Also, to understand how Prolog can use backtracking to find multiple ways to insert the item at different positions automatically.

Theory

In normal programming languages, inserting an item into the middle of an array means using a loop to shift all the other elements over to make room. In Prolog, we don't change the original list; we build a brand new one instead.

To do this, we write two rules. The first rule is the simplest scenario: we just take the new item and attach it directly to the very front of the list using the head-tail operator.

The second rule handles putting the item somewhere in the middle or at the end. It splits the list, leaves the Head alone, and recursively tries to insert the new item into the Tail. Once the recursion finishes, it attaches the Head back onto the front. Because Prolog searches for all possible truths, if we ask for multiple answers, it will try the first rule, then backtrack and try the second rule over and over. This means it will actually generate every single possible list where the new item is placed in a different spot!

Code

```
insert_element(X, List, [X|List]).  
insert_element(X, [H|T], [H|Result]) :- insert_element(X, T, Result).
```

Query and Output

```
?- insert_element(99, [1, 2, 3], Result).  
Result = [99, 1, 2, 3] ;  
Result = [1, 99, 2, 3] ;  
Result = [1, 2, 99, 3] ;  
Result = [1, 2, 3, 99].
```

Conclusion

In this experiment, I successfully wrote a Prolog program to insert an element into a list. I learned how to use two simple rules to either put the item at the front or recursively push it deeper into the list. Pressing the semicolon in the terminal showed me how Prolog's automatic backtracking easily generates all possible insertion points without needing any complex loops or index numbers.

Experiment No: 16

Experiment name: Write a LISP program applying basic LIST manipulation function and verify their operation.

Objectives

To learn how to work with lists in LISP using its built-in commands. Also, to verify how these specific functions can extract parts of a list, add new items, combine multiple lists together, and check their size.

Theory

LISP actually stands for "LISt Processing", so handling lists is what it does best. Unlike Prolog, which uses pattern matching to split lists, LISP uses specific built-in functions to do the heavy lifting.

To take a list apart, we use two main commands: 'car' grabs the very first item (the head), and 'cdr' grabs the rest of the list (the tail).

To put lists together, we use 'cons' to push a single new item onto the absolute front of a list. If we have two completely separate lists and want to merge them into one long list, we use 'append'. LISP also provides easy utility functions like 'length' to count how many items are inside, and 'reverse' to flip the entire list backwards. In this program, we will define a test list and print the results of these operations to verify their behavior.

Code

```
(setq my-list '(a b c d))
(setq list-two '(e f))
(print (car my-list))
(print (cdr my-list))
(print (cons 'z my-list))
(print (append my-list list-two))
(print (length my-list))
(print (reverse my-list))
```

Query and Output

```
A
(B C D)
(Z A B C D)
(A B C D E F)
4
(D C B A)
```

Conclusion

In this experiment, I successfully wrote a LISP program to manipulate basic lists. I learned how to use built-in functions like 'car', 'cdr', 'cons', and 'append' to modify and

read data. This verified that working with lists in LISP is incredibly easy and direct because the language provides ready-to-use commands for almost any list operation you might need without requiring manual recursion.

Experiment No: 17

Experiment name: Write a LISP program applying additional LIST manipulation function and verify their operation.

Objectives

To explore more advanced list manipulation commands in LISP. Also, to learn how to instantly find specific items, check if an item exists inside a list, and build lists directly from individual elements.

Theory

In the previous experiment, we looked at the absolute basics like grabbing the head or tail of a list. But sometimes we need to search a list or jump straight to a specific spot without doing it manually step-by-step. LISP has built-in functions for this too.

Instead of just using the quote symbol to make a list, we can use the ‘list’ function. It takes a bunch of separate items and instantly wraps them into a new list.

If we want to grab a specific item from the middle of the list, we use ‘nth’. We just give it an index number (starting from 0) and the list, and it pulls out that exact item. If we just want the very end of the list, ‘last’ jumps straight to the final item (though it returns it inside a small list).

Finally, ‘member’ acts like a search tool. We tell it what to look for, and if it finds the item, it returns the rest of the list starting from that exact item. If the item isn’t anywhere in the list, it returns ‘NIL’, which is LISP’s version of false.

Code

```
(setq my-list (list 'apple 'banana 'cherry 'date))
(print my-list)
(print (last my-list))
(print (nth 2 my-list))
(print (member 'cherry my-list))
(print (member 'grape my-list))
```

Query and Output

```
(APPLE BANANA CHERRY DATE)
(DATE)
CHERRY
(CHERRY DATE)
NIL
```

Conclusion

In this experiment, I successfully wrote a LISP program using additional list manipulation functions. I learned how to use ‘list’ to group individual items together, ‘nth’ and ‘last’ to

quickly extract specific elements without writing recursive rules, and ‘member‘ to search for data. This proved that LISP has a very rich set of built-in tools that make navigating and searching complex lists incredibly fast and easy.

Experiment No: 18

Experiment name: Write a LISP program to calculate the maximum and minimum of some numbers.

Objectives

To learn how to find the largest and smallest numbers in LISP. Also, to understand how to use built-in math functions on multiple arguments and how to apply them to an entire list.

Theory

In many programming languages, finding the maximum or minimum number means writing a loop that goes through an array and compares each number one by one against a temporary variable. LISP makes this much easier because it is designed to handle multiple arguments at once.

LISP has built-in math functions called ‘max’ and ‘min’. You can simply pass a bunch of individual numbers directly to these functions, and they will instantly return the highest or lowest value.

If your numbers are already stored inside a list variable, you can’t just pass the list directly to ‘max’ because it expects separate numbers, not a single list object. To fix this, we use the ‘apply’ function. ‘apply’ takes a function (like ‘max’ or ‘min’) and unpacks the list so the function can read all the individual items inside it perfectly.

Code

```
(setq my-numbers '(45 12 89 33 7))  
(print (apply 'max my-numbers))  
(print (apply 'min my-numbers))  
(print (max 15 100 2))  
(print (min 15 100 2))
```

Query and Output

```
89  
7  
100  
2
```

Conclusion

In this experiment, I successfully wrote a LISP program to calculate the maximum and minimum of numbers. I learned how to use the built-in ‘max’ and ‘min’ commands directly on separate numbers. Using the ‘apply’ function also showed me how to unpack a list and feed its contents into mathematical functions, completely avoiding the need for any manual comparison loops.

Experiment No: 19

Experiment name: Write a LISP program using PutProp function to get different properties of an object.

Objectives

To learn how to attach extra information to an object in LISP using property lists. Also, to understand how to use the **putprop** command to save specific characteristics and the **get** command to read them back.

Theory

In regular programming, if we want an object to have different properties (like a student having a name, age, and department), we usually have to create a complex class, struct, or dictionary. LISP makes this much simpler using something called a "property list."

Every symbol (or object name) in LISP has a hidden list attached to it where we can store custom data. The **putprop** function is used to put a property into this hidden list. We just give it three things: the name of the object, the value we want to save, and the label (or indicator) for that property.

Once the properties are attached to the object, they stay there in the background. Whenever we need to look up a specific piece of information, we use the **get** function. We just tell it the object's name and the property label we are looking for, and it instantly pulls out the correct value without us having to search through any arrays or lists manually.

Code

```
(putprop 'student 'nobin 'name)
(putprop 'student 22 'age)
(putprop 'student 'cse 'department)
(print (get 'student 'name))
(print (get 'student 'age))
(print (get 'student 'department))
(print (get 'student 'blood_group))
```

Query and Output

```
NOBIN
22
CSE
NIL
```

Conclusion

In this experiment, I successfully wrote a LISP program to manage object properties using **putprop**. I learned how to assign different pieces of data to a single symbol and how to retrieve them instantly using **get**. I also saw that asking for a property that

hasn't been set yet just returns `NIL`, proving that property lists are a very safe and easy way to handle object characteristics without needing complex data structures.

Experiment No: 20

Experiment name: Write a C++ program to implement Breadth First Search algorithm.

Objectives

To learn how to explore a graph using the Breadth-First Search (BFS) method. Also, to understand how a queue data structure is used to visit all the closest neighbors of a node before moving deeper into the graph.

Theory

In computer science, a graph is a bunch of nodes connected by edges. Breadth-First Search (BFS) is a way to explore this graph level by level. Imagine dropping a stone in water; the ripples spread outwards. BFS works exactly like that.

To code this in C++, we use a 'queue'. We start at a specific node, mark it as visited so we don't count it twice, and push it into the queue. Then, we enter a loop where we take the node at the front of the queue, print it out, and look at all of its direct neighbors. If a neighbor hasn't been visited yet, we mark it as visited and push it to the back of the queue. We keep doing this until the queue is completely empty. This guarantees that we check all the nodes one step away before we check any nodes that are two steps away.

Code

```
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

void bfs(int startNode, vector<vector<int>>& adjList, vector<bool>& visited) {
    queue<int> q;

    q.push(startNode);
    visited[startNode] = true;

    while (!q.empty()) {
        int currentNode = q.front();
        q.pop();
        cout << currentNode << " ";

        for (int neighbor : adjList[currentNode]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
}
```

```

}

int main() {
    int totalNodes = 5;
    vector<vector<int>> adjList(totalNodes);
    vector<bool> visited(totalNodes, false);

    adjList[0].push_back(1);
    adjList[0].push_back(2);

    adjList[1].push_back(0);
    adjList[1].push_back(3);
    adjList[1].push_back(4);

    adjList[2].push_back(0);

    adjList[3].push_back(1);

    adjList[4].push_back(1);

    cout << "BFS traversal starting from node 0: ";
    bfs(0, adjList, visited);
    cout << endl;

    return 0;
}

```

Output

BFS traversal starting from node 0: 0 1 2 3 4

Conclusion

In this experiment, I successfully implemented the Breadth-First Search algorithm using C++. I learned how to represent a graph using an adjacency list and how to manage the traversal using a standard queue. This showed me practically how BFS visits nodes level by level, ensuring that we explore the widest parts of the graph before diving deeper.

Experiment No: 21

Experiment name: Write a C++ program to implement Depth First Search algorithm.

Objectives

To learn how to explore a graph using the Depth-First Search (DFS) method. Also, to understand how to use recursion to go as deep as possible along a single branch before backing up to check other paths.

Theory

In the last experiment with BFS, we checked all the closest neighbors first, like ripples in a pond. DFS is the complete opposite. Imagine walking through a maze: you pick a path and keep walking forward until you hit a dead end. Only when you get stuck do you turn around, take a step back, and try a different hallway.

To write this in C++, we usually use recursion, which naturally acts like a memory stack. We start at a node, mark it as visited so we don't get stuck in a loop, and print it out. Then, we look at its neighbors. The moment we find a neighbor that hasn't been visited, we instantly call the DFS function on that new node, diving deeper into the graph. The program keeps plunging forward until it runs out of unvisited neighbors, at which point the recursion automatically backtracks to explore any branches it left behind.

Code

```
#include <iostream>
#include <vector>

using namespace std;

void dfs(int currentNode, vector<vector<int>>& adjList, vector<bool>&
    visited) {
    visited[currentNode] = true;
    cout << currentNode << " ";

    for (int neighbor : adjList[currentNode]) {
        if (!visited[neighbor]) {
            dfs(neighbor, adjList, visited);
        }
    }
}

int main() {
    int totalNodes = 5;
    vector<vector<int>> adjList(totalNodes);
    vector<bool> visited(totalNodes, false);

    adjList[0].push_back(1);
    adjList[0].push_back(2);
```

```
adjList[1].push_back(0);
adjList[1].push_back(3);
adjList[1].push_back(4);

adjList[2].push_back(0);

adjList[3].push_back(1);

adjList[4].push_back(1);

cout << "DFS traversal starting from node 0: ";
dfs(0, adjList, visited);
cout << endl;

return 0;
}
```

Output

DFS traversal starting from node 0: 0 1 3 4 2

Conclusion

In this experiment, I successfully implemented the Depth-First Search algorithm using C++. I learned how to use a simple recursive function to navigate through a graph's connections. This practically showed me how DFS prioritizes exploring one complete path to its very end before backtracking, proving it to be a great approach for tasks like solving mazes or finding deep connections in data.

Experiment No: 22

Experiment name: Write a C++ program to implement A* Search algorithm.

Objectives

To learn how the A* search algorithm finds the shortest path in a graph. Also, to understand how to combine the actual distance traveled (g-cost) and an educated guess to the finish line (h-cost) to make smart movement decisions.

Theory

In previous experiments, Breadth-First Search explored everywhere equally, which can be very slow. A* (A-star) is a much smarter algorithm because it doesn't just guess blindly; it uses a map.

A* calculates a score for every possible step using the formula $f = g + h$.

- **g-cost:** The actual exact distance it took to get from the starting point to the current node.
- **h-cost (Heuristic):** A smart guess of how far it is from the current node to the final goal.

The algorithm adds these two numbers together to get the total f score. In C++, we use a priority-queue to always sort our choices so that the node with the lowest f score stays at the top. The program always picks this lowest-score node to explore next, skipping bad paths and heading straight toward the target as efficiently as possible.

Code

```
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

struct Node {
    int id, f, g;
    bool operator>(const Node& other) const {
        return f > other.f;
    }
};

void aStar(int start, int goal, vector<vector<pair<int, int>>>& adj,
vector<int>& h) {
    priority_queue<Node, vector<Node>, greater<Node>> pq;
    vector<int> g(adj.size(), 10000);
    vector<int> parent(adj.size(), -1);

    pq.push({start, h[start], 0});
```

```

g[start] = 0;

while (!pq.empty()) {
    int curr = pq.top().id;
    pq.pop();

    if (curr == goal) {
        cout << "Path found. Cost: " << g[goal] << "\nPath: ";
        vector<int> path;
        for (int v = goal; v != -1; v = parent[v]) {
            path.push_back(v);
        }
        for (int i = path.size() - 1; i >= 0; i--) {
            cout << path[i] << " ";
        }
        cout << endl;
        return;
    }

    for (auto edge : adj[curr]) {
        int next = edge.first;
        int weight = edge.second;
        int newG = g[curr] + weight;

        if (newG < g[next]) {
            g[next] = newG;
            parent[next] = curr;
            pq.push({next, newG + h[next], newG});
        }
    }
}

cout << "No path found." << endl;
}

int main() {
    int nodes = 5;
    vector<vector<pair<int, int>>> adj(nodes);
    vector<int> h = {7, 6, 2, 1, 0};

    adj[0].push_back({1, 2});
    adj[0].push_back({2, 5});
    adj[1].push_back({2, 4});
    adj[1].push_back({3, 6});
    adj[2].push_back({3, 2});
    adj[3].push_back({4, 3});

    aStar(0, 4, adj, h);

    return 0;
}

```

Output

```
Path found. Cost: 10  
Path: 0 2 3 4
```

Conclusion

In this experiment, I successfully implemented the A* Search algorithm in C++. I learned how to use a priority queue to sort and pick the most promising node based on its combined f-cost. It showed me how adding a heuristic dramatically speeds up pathfinding because it actively guides the program toward the destination instead of just searching in every random direction.